

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0518

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 20%

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

2. OCR Pipeline for Document Processing

A document processing system must:

- A. Read input images containing printed text
- B. Enhance image quality (noise removal, contrast improvement)
- C. Segment text regions
- D. Extract and display recognized text

Constraints:

- Input images may be noisy and low contrast
- Batch processing required

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

OCR Pipeline for Document Processing

Objective: Design an end-to-end OCR pipeline using OpenCV to handle noisy, low-contrast document images, segment text regions, and extract recognized text in batch mode.

Pseudocode

BEGIN OCR_PIPELINE

INPUT:

Folder containing multiple document images

FOR each image_file in input_folder DO

STAGE 1: IMAGE INPUT

image ← cv2.imread(image_file)

IF image is NULL THEN

 Print "Unable to read image"

 CONTINUE

END IF

STAGE 2: PREPROCESSING

gray ← cv2.cvtColor(image, COLOR_BGR2GRAY)

Noise removal

denoised ← cv2.GaussianBlur(gray, (5,5), 0)

Contrast improvement

enhanced ← cv2.createCLAHE(
 clipLimit = 2.0,
 tileGridSize = (8,8)
).apply(denoised)

Convert image to binary

binary ← cv2.threshold(
 enhanced,
 0,
 255,
 THRESH_BINARY + THRESH_OTSU
)

```

# Remove small noise
kernel ← cv2.getStructuringElement(
    MORPH_RECT, (3,3)
)
clean ← cv2.morphologyEx(
    binary,
    MORPH_OPEN,
    kernel
)

```

STAGE 3: TEXT REGION SEGMENTATION

```

# Detect text contours
contours ← cv2.findContours(
    clean,
    RETR_EXTERNAL,
    CHAIN_APPROX_SIMPLE
)

text_regions ← EMPTY LIST
FOR each contour in contours DO
    x, y, width, height ← cv2.boundingRect(contour)
    # Ignore very small components
    IF width > MIN_WIDTH AND height > MIN_HEIGHT THEN
        region ← image[y:y+height, x:x+width]

        ADD (x, y, width, height, region)
        TO text_regions

    END IF

END FOR

# Sort regions from top-to-bottom and left-to-right
SORT text_regions according to y-coordinate, x-coordinate

```

STAGE 4: TEXT EXTRACTION

recognized_text ← EMPTY STRING

FOR each region in text_regions DO

 # Prepare individual text region

 region_gray ← cv2.cvtColor(
 region,
 COLOR_BGR2GRAY
)

 region_binary ← cv2.threshold(
 region_gray,
 0,
 255,
 THRESH_BINARY + THRESH_OTSU
)

 # OCR using Tesseract

 text ← pytesseract.image_to_string(
 region_binary
)

 recognized_text ←
 recognized_text + text

END FOR

STAGE 5: OUTPUT

Print "Recognized Text:"

Print recognized_text

Save recognized_text to output_file

END FOR

END OCR_PIPELINE

Key point: CLAHE is particularly useful for the given **low-contrast constraint**, while Gaussian filtering and morphological opening address **noise**. Otsu thresholding provides automatic binarization, and contour-based segmentation isolates candidate text regions before OCR.

5. Facial Expression Recognition System

A mobile-based system must:

- A. Capture images from a camera
- B. Detect faces
- C. Extract facial features
- D. Classify expressions (happy, sad, neutral)
- E. Display results

Constraints:

- Limited computational resources
- Real-time response required

Write pseudocode for the pipeline, ensuring efficient processing and modular design for detection, feature extraction, and classification.

Facial Expression Recognition System

Pseudocode

BEGIN FACIAL_EXPRESSION_RECOGNITION

INITIALIZE:

 Camera

 Face Detector

 Feature Extractor

 Expression Classifier

WHILE camera is running DO

 MODULE 1: IMAGE CAPTURE

 frame ← Capture frame from camera

 IF frame is NULL THEN

 CONTINUE

 END IF

 # Reduce computational cost

```
small_frame ← Resize(frame, 50% or lower)
```

MODULE 2: FACE DETECTION

```
gray ← ConvertToGrayscale(small_frame)
```

```
faces ← DetectFaces(  
    gray,  
    scaleFactor = 1.1,  
    minNeighbors = 5  
)
```

```
FOR each face in faces DO
```

```
    x, y, width, height ← FaceBoundingBox(face)  
    face_image ← Crop(  
        gray,  
        x, y, width, height  
    )
```

MODULE 3: PREPROCESSING

```
face_image ← Resize(face_image, 48 × 48)
```

```
face_image ← NormalizeIntensity(face_image)
```

MODULE 4: FEATURE EXTRACTION

```
features ← ExtractFeatures(face_image)
```

```
# Lightweight options:
```

```
# HOG / Local Binary Patterns (LBP)
```

```
# or a lightweight CNN feature extractor
```

MODULE 5: EXPRESSION CLASSIFICATION

```
expression ← Classifier.predict(features)
```

```
IF expression = HAPPY THEN
    result ← "Happy"
ELSE IF expression = SAD THEN
    result ← "Sad"
```

```
ELSE
    result ← "Neutral"
END IF
```

```
-----
MODULE 6: DISPLAY RESULT
-----
```

```
DrawRectangle(
    frame,
    x, y, width, height
)
```

```
PutText(
    frame,
    result,
    x, y - 10
)
```

```
END FOR
```

```
Display(frame)
```

```
# Stop when user presses ESC
```

```
IF KeyPressed = ESC THEN
```

```
    BREAK
```

```
END IF
```

```
END WHILE
```

```
Release camera
```

```
Close all windows
```

```
END FACIAL_EXPRESSION_RECOGNITION
```

Result: The modular pipeline efficiently captures camera frames, detects faces, extracts compact facial features, classifies them into **happy, sad, or neutral**, and displays the result in real time.

9. Background Subtraction Pipeline

A surveillance system must:

- A. Read video stream
- B. Separate foreground from background
- C. Highlight moving objects

Constraints:

- Handle dynamic background
- Reduce noise

Write pseudocode including background modeling, subtraction, filtering, and display.

Background Subtraction Pipeline

Pseudocode

BEGIN BACKGROUND_SUBTRACTION

INITIALIZE:

VideoCapture()

BackgroundModel $\leftarrow$ MOG2

MorphologicalKernel $\leftarrow 3 \times 3$

WHILE video is running DO

STAGE 1: VIDEO INPUT

frame $\leftarrow$ Read frame from video


```
IF frame is NULL THEN
```

```
    BREAK
```

```
END IF
```

```
-----
```

```
STAGE 2: BACKGROUND MODELING
```

```
-----
```

```
foreground_mask ← BackgroundModel.apply(
```

```
    frame
```

```
)
```

```
# MOG2 automatically updates the background model
```

```
# and adapts to gradual changes in the scene.
```

```
-----
```

```
STAGE 3: NOISE REDUCTION
```

```
-----
```

```
# Remove shadows and weak detections
```

```
foreground_mask ← Threshold(
```

```
    foreground_mask,
```

```
    threshold_value
```

```
)
```

```
# Remove small isolated noise
```

```
foreground_mask ← MorphologicalOpening(
```

```
        foreground_mask,  
  
        MorphologicalKernel  
    )
```

Fill small gaps inside moving objects

```
foreground_mask ← MorphologicalClosing(  
  
    foreground_mask,  
  
    MorphologicalKernel  
)
```

STAGE 4: MOVING OBJECT DETECTION

```
contours ← FindContours(  
  
    foreground_mask  
)  
  
FOR each contour in contours DO  
  
    x, y, width, height ← BoundingBox(contour)  
  
    area ← CalculateArea(contour)  
  
    IF area > MIN_OBJECT_AREA THEN  
  
        DrawRectangle(  
  
            frame,
```

```
        x, y, width, height
    )

    DrawLabel(

        frame,

        "Moving Object",

        x, y

    )

END IF

END FOR

-----

STAGE 5: DISPLAY

-----

Display("Original Video", frame)

Display(

    "Foreground Mask",

    foreground_mask

)

IF ESC key is pressed THEN

    BREAK

END IF

END WHILE
```

Release video

Destroy all windows

END BACKGROUND_SUBTRACTION

10. Edge Detection Pipeline for Real-Time Video

A system must:

- A. Capture video frames
- B. Apply edge detection
- C. Display results

Constraints:

- Real-time processing
- Maintain clarity of edges

Write pseudocode with preprocessing, edge detection, and visualization modules.

Edge Detection Pipeline for Real-Time Video

Pseudocode

BEGIN EDGE_DETECTION_PIPELINE

INITIALIZE:

Camera $\leftarrow$ cv2.VideoCapture(0)

GaussianKernel $\leftarrow$ (5,5)

CannyThreshold1 $\leftarrow$ 50

CannyThreshold2 $\leftarrow$ 150

WHILE Camera is running DO

MODULE 1: VIDEO CAPTURE

```
frame ← Camera.read()
```

```
IF frame is NULL THEN
```

```
    BREAK
```

```
END IF
```

MODULE 2: PREPROCESSING

```
# Convert color image to grayscale
```

```
gray ← cv2.cvtColor(
```

```
    frame,
```

```
    COLOR_BGR2GRAY
```

```
)
```

```
# Reduce noise while preserving edges
```

```
smooth ← cv2.GaussianBlur(
```

```
    gray,
```

```
    GaussianKernel,
```

```
    0
```

```
)
```

MODULE 3: EDGE DETECTION

```
edges ← cv2.Canny(  
    smooth,  
    CannyThreshold1,  
    CannyThreshold2  
)
```

MODULE 4: EDGE ENHANCEMENT

```
# Optional morphological operation  
kernel ← cv2.getStructuringElement(  
    MORPH_RECT,  
    (3,3)  
)  
edges ← cv2.morphologyEx(  
    edges,  
    MORPH_CLOSE,  
    kernel  
)
```

MODULE 5: VISUALIZATION

Display(

 "Original Video",

 frame

)

Display(

 "Detected Edges",

 edges

)

IF ESC key is pressed THEN

 BREAK

END IF

END WHILE

Release Camera

Destroy all Windows

END EDGE_DETECTION_PIPELINE

12. Face Detection with ROI Extraction

A system must:

- A. Detect faces in an image
- B. Extract face regions
- C. Save cropped faces

Constraints:

- Handle multiple faces
- Avoid redundant processing

Write pseudocode including detection, ROI extraction, and saving logic.

Face Detection with ROI Extraction

Pseudocode

BEGIN FACE_DETECTION_ROI

INPUT:

Image containing one or more faces

Output folder for cropped faces

INITIALIZE:

FaceDetector $\leftarrow$ Load Haar Cascade / Face Detector

FaceCounter $\leftarrow$ 0

MODULE 1: IMAGE INPUT

image $\leftarrow$ cv2.imread(input_image)

IF image is NULL THEN

Print "Image cannot be loaded"

STOP

END IF

MODULE 2: PREPROCESSING

gray $\leftarrow$ cv2.cvtColor(


```
        image,  
  
        COLOR_BGR2GRAY  
  
    )
```

MODULE 3: FACE DETECTION

```
faces ← FaceDetector.detectMultiScale(  
  
    gray,  
  
    scaleFactor = 1.1,  
  
    minNeighbors = 5,  
  
    minSize = (30,30)  
  
    )
```

IF faces is EMPTY THEN

 Print "No faces detected"

 STOP

END IF

MODULE 4: ROI EXTRACTION

FOR each face in faces DO

 x, y, width, height ← face

```
# Extract only the detected face region
```

```
face_ROI ← image[  
    y : y + height,  
    x : x + width  
]
```

MODULE 5: AVOID REDUNDANT PROCESSING

```
# Ignore very small or invalid regions
```

```
IF width < MIN_WIDTH OR height < MIN_HEIGHT THEN
```

```
    CONTINUE
```

```
END IF
```

```
# Process each detected bounding box only once
```

```
FaceCounter ← FaceCounter + 1
```

MODULE 6: SAVE CROPPED FACE

```
filename ← "face_" + FaceCounter + ".jpg"
```

```
output_path ← JoinPath(  
    output_folder,  
    filename
```

```

        )

    cv2.imwrite(

        output_path,

        face_ROI

    )

END FOR

-----

MODULE 7: OUTPUT

-----

Print FaceCounter + " faces detected and saved"

END FACE_DETECTION_ROI

```

Result: The pipeline detects all valid faces, extracts their **ROI (Region of Interest)** independently, and saves each cropped face with a unique filename while avoiding unnecessary processing.

Code of Conduct:

*I P.V.N. ANISH BABU (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: P.V.N. ANISH